

CNN-Based Traffic Sign Recognition System

ASSIGNMENT PROJECT REPORT

Submitted in the partial fulfilment for the Course of

MLA0408 – Deep Learning

to the award of the

degree of

**BACHELOR OF
ENGINEERING IN**

CSE

Submitted by

D. Bindu Priya (192425162)

K. Gowthami (192525428)

M. Sri Lakshmi (192525404)

Under the Supervision of

Dr. D. Sudhagar



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

**Saveetha Institute of Medical and
Technical Sciences Chennai-602105**

September 2026

CNN-Based Traffic Sign Recognition System

Deep Learning (MLA0408) — Course Assignment Report

A. Assignment Information

Department	CSE
Programme	B.Tech
Course Code & Name	MLA0408 – Deep Learning
Academic Year / Batch	2026–27
Faculty Name	Dr. D. Sudhagar
Assignment Title	CNN-Based Traffic Sign Recognition System
Date of Issue	01.09.2026
Date of Submission	03.09.2026
Maximum Marks	100
Team Size	4 members

Team Members:

D. Bindu Priya (192425162) – Dataset & Preprocessing Pipeline

K. Gowthami (192525428) – CNN Architecture & Model Training

M. Srilakshmi (192525404) – Evaluation, Metrics & Visualization

1. Abstract

Traffic signs play an important role in maintaining road safety and guiding drivers. Automatic traffic sign recognition can help intelligent transportation systems understand road signs quickly and accurately. In this project, a **Convolutional Neural Network (CNN)** is used to recognize different types of traffic signs from images. The proposed system takes a traffic sign image as input, preprocesses the image, and passes it through a CNN model. The CNN automatically learns important features such as edges, shapes, colors, and patterns from the images. Finally, the model classifies the input image into the appropriate traffic sign category. This system can be useful in autonomous vehicles, driver-assistance systems, and smart transportation applications.

Keywords: CNN, Deep Learning, Traffic Sign Recognition, Image Classification, Computer Vision, Autonomous Vehicles.

2. Introduction

Traffic signs provide important information to drivers, such as speed limits, warnings, directions, and restrictions. Correctly identifying these signs is essential for safe driving. However, recognizing traffic signs manually can be difficult in situations such as poor lighting, bad weather, or when the vehicle is moving at high speed.

Deep Learning provides effective solutions for image recognition problems. Among different deep learning techniques, **Convolutional Neural Networks (CNNs)** are widely used for image classification. CNNs can automatically extract useful features from images without requiring manual feature selection.

In this assignment, a CNN-based system is designed to recognize traffic signs. The model is trained using a dataset containing different categories of traffic sign images. After training, the model can predict the class of a new traffic sign image.

3. Problem Statement

Traffic signs are essential for maintaining road safety and providing information to drivers. However, manually identifying traffic signs can be difficult due to factors such as poor lighting, weather conditions, image distortion, and the high speed of moving vehicles. Therefore, there is a need for an automated system that can accurately recognize and classify traffic signs from images.

The CNN-Based Traffic Sign Recognition System aims to develop a deep learning model using a Convolutional Neural Network (CNN) to automatically identify different types of traffic signs. The system processes input images, extracts important visual features, and classifies them into the correct traffic sign categories. This system can support driver-assistance systems, autonomous vehicles, and intelligent transportation applications, thereby improving road safety and reducing human error. Develop a deep learning-based system that can automatically identify and classify traffic signs from input images using a **Convolutional Neural Network (CNN)**.

The system should:

1. Accept traffic sign images as input.
2. Preprocess the images.
3. Extract important image features using CNN layers.
4. Train the CNN model using labeled traffic sign images.
5. Classify unseen traffic sign images.

- Evaluate the performance of the trained model.

4. Objectives

The main objectives of the project are:

- To understand the application of CNN in image classification.
- To preprocess traffic sign images for deep learning.
- To build a CNN model for traffic sign recognition.
- To train the model using labeled traffic sign data.
- To test the model using unseen images.
- To measure model performance using accuracy and other evaluation metrics.
- To understand the role of deep learning in intelligent transportation systems.

5. Proposed System

The proposed system uses a CNN to classify traffic sign images.

The overall process is:

Input Image → Preprocessing → CNN Feature Extraction → Classification → Predicted Traffic Sign

The input traffic sign image is first resized into a fixed dimension and normalized. The processed image is then given to the CNN. Convolution and pooling layers extract important features from the image. The extracted features are passed to fully connected layers, which classify the image into one of the traffic sign categories.

6. Dataset

For this project, a traffic sign image dataset such as the **German Traffic Sign Recognition Benchmark (GTSRB)** can be used.

The dataset contains images belonging to multiple traffic sign categories. Each image has a corresponding class label.

Dataset characteristics

- Type: Image classification dataset
- Input: Traffic sign images
- Output: Traffic sign class
- Images: Different sizes and conditions
- Classes: Multiple traffic sign categories
- Application: Traffic sign recognition

The dataset can be divided into:

Dataset	Purpose
Training Dataset	Used to train the CNN
Validation Dataset	Used to monitor model performance during training
Testing Dataset	Used to evaluate the final model

A commonly used division is approximately **70% training, 15% validation, and 15% testing**.

7. Data Preprocessing

Before training the CNN, the images need to be preprocessed.

Main preprocessing steps:

1. **Image Resizing:** All images are resized to the same dimensions, such as 32×32 pixels.
2. **Normalization:** Pixel values are converted from the range 0–255 to a smaller range such as 0–1.
3. **Label Encoding:** Each traffic sign category is converted into a numerical class label.
4. **Data Augmentation:** Images can be rotated, shifted, zoomed, or flipped slightly to increase the diversity of the training data.
5. **Dataset Splitting:** The dataset is divided into training, validation, and testing sets.

8. CNN Architecture

The CNN consists of several layers that perform different operations.

Proposed Architecture

Input Image ($32 \times 32 \times 3$)

↓

Convolution Layer

↓

ReLU Activation

↓

Max Pooling

↓

Convolution Layer

↓

ReLU Activation

↓

Max Pooling

↓

Flatten Layer

↓

Fully Connected Layer

↓

Dropout

↓

Output Layer – Softmax

Explanation of Layers

1. Convolution Layer:

Extracts important features such as edges, curves, shapes, and patterns from the traffic sign images.

2. ReLU Activation:

Introduces non-linearity into the network and helps the model learn complex patterns.

3. Max Pooling:

Reduces the spatial size of feature maps while retaining important features.

4. Flatten Layer:

Converts the extracted feature maps into a one-dimensional vector.

5. Fully Connected Layer:

Uses the extracted features for classification.

6. Dropout Layer:

Reduces overfitting by randomly disabling some neurons during training.

7. Softmax Output Layer:

Produces probabilities for all traffic sign classes. The class with the highest probability is selected as the prediction.

9. Working Principle

The working of the proposed system can be explained in the following steps:

Step 1: Input

A traffic sign image is provided to the system.

Step 2: Preprocessing

The image is resized and normalized so that it can be processed by the CNN.

Step 3: Feature Extraction

Convolutional layers identify important features such as edges, colors, shapes, and patterns.

Step 4: Feature Reduction

Pooling layers reduce the size of feature maps and retain the most important information.

Step 5: Classification

The extracted features are passed through fully connected layers.

Step 6: Prediction

The softmax layer calculates the probability of each traffic sign class.

Step 7: Output

The class with the highest probability is displayed as the recognized traffic sign.

10. Algorithm

CNN-Based Traffic Sign Recognition Algorithm

Input: Traffic sign image dataset

Output: Predicted traffic sign class

1. Collect the traffic sign image dataset.
2. Resize all images to a fixed size.
3. Normalize the image pixel values.
4. Divide the dataset into training, validation, and testing sets.
5. Create the CNN architecture.
6. Apply convolution and ReLU activation.
7. Apply pooling to reduce feature dimensions.
8. Repeat convolution and pooling for feature extraction.
9. Flatten the extracted features.
10. Pass the features through fully connected layers.
11. Apply softmax activation at the output layer.
12. Train the CNN using the training dataset.

13. Validate the model using validation data.
14. Test the trained model using test images.
15. Calculate accuracy and other performance measures.
16. Display the predicted traffic sign.

11. Flowchart

You can create the following flowchart in Word:

Start



Collect Traffic Sign Dataset



Preprocess Images



Resize and Normalize Images



Split Dataset



Build CNN Model



Train CNN Model



Validate Model



Test with New Image



Predict Traffic Sign Class



Display Result



End

12. Training the CNN Model

The CNN model is trained using labeled traffic sign images. During training, the model compares its predicted output with the actual class label and calculates the error using a loss function.

The model weights are updated using the **Adam optimizer** or another suitable optimization algorithm.

A suitable configuration can be:

Parameter	Value
Image Size	$32 \times 32 \times 3$
Number of Classes	Dataset dependent
Optimizer	Adam
Activation	ReLU
Output Activation	Softmax
Loss Function	Categorical Cross-Entropy
Batch Size	32

Parameter	Value
Epochs	20–30
Validation	Yes

These values can be adjusted depending on the dataset and computational resources.

13. Loss Function

For a multi-class traffic sign classification problem, **categorical cross-entropy** can be used as the loss function.

The loss function measures the difference between the actual class and the predicted class. A lower loss value indicates that the model's predictions are closer to the actual labels.

14. Performance Evaluation

The performance of the CNN model can be evaluated using several metrics.

1. Accuracy

Accuracy represents the percentage of correctly classified images.

$$Accuracy = \frac{Correct\ Predictions}{Total\ Predictions} \times 100$$

2. Precision

Precision measures how many of the images predicted as a particular class actually belong to that class.

3. Recall

Recall measures how many actual images of a particular class were correctly identified.

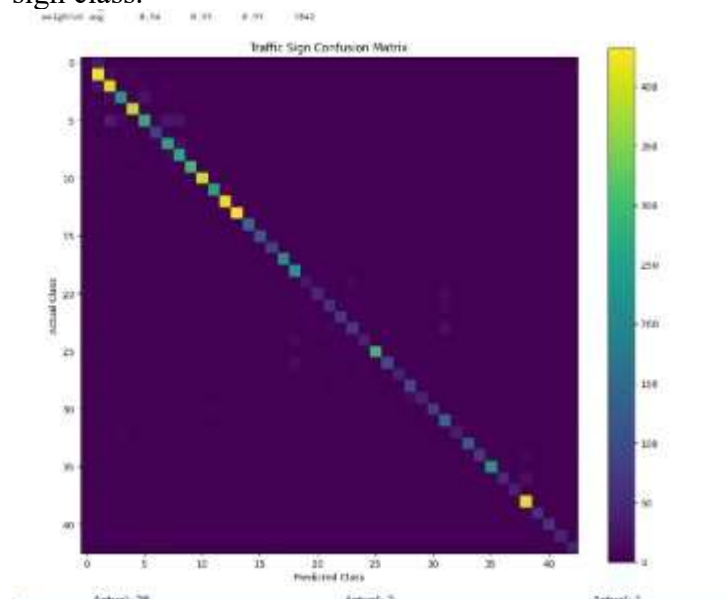
4. F1-Score

F1-score provides a balance between precision and recall.

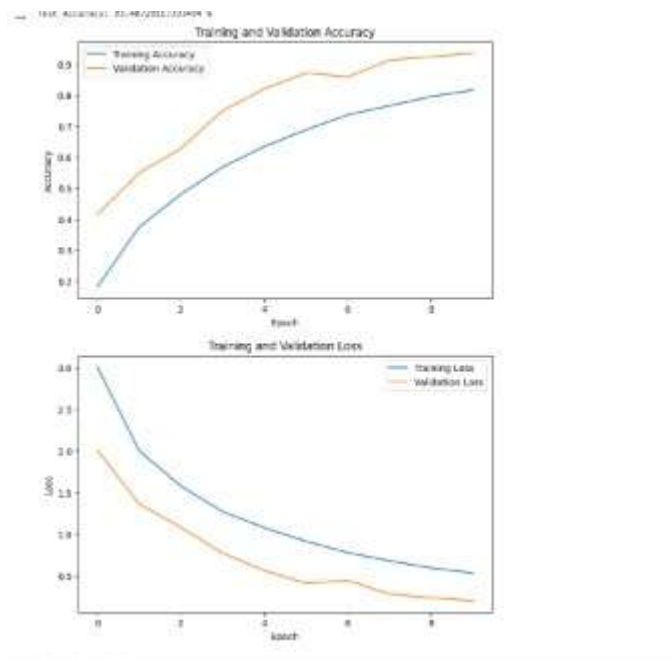
$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

5. Confusion Matrix

A confusion matrix shows the number of correct and incorrect predictions for each traffic sign class.



15. Expected Results



After training, the CNN model should be able to identify different traffic signs from test images.

For example:

Input Traffic Sign	Expected Prediction
Speed Limit Sign	Speed Limit
Stop Sign	Stop
No Entry Sign	No Entry
Pedestrian Crossing	Pedestrian Crossing
Yield Sign	Yield
Right Turn Sign	Right Turn

The training and validation accuracy can be plotted using graphs. A confusion matrix can also be generated to understand which classes are correctly classified and which classes are confused with each other.

Note: The actual accuracy should be reported only after running the model on the selected dataset.

16. Advantages

The proposed CNN-based traffic sign recognition system has the following advantages:

- Automatic traffic sign recognition.
- Reduces the need for manual identification.
- CNN automatically extracts image features.
- Can recognize multiple traffic sign categories.
- Useful for real-time intelligent transportation systems.
- Can support driver-assistance systems.

- Can be integrated with autonomous vehicle technologies.
- Deep learning models can achieve high classification performance when trained with suitable data.

17. Limitations

Some limitations of the system are:

- Performance depends on the quality and size of the dataset.
- Poor lighting can affect recognition.
- Blurred images may reduce accuracy.
- Occluded or partially visible signs can be difficult to recognize.
- Training CNN models requires computational resources.
- Similar-looking traffic signs may sometimes be misclassified.

18. Applications

CNN-based traffic sign recognition can be used in:

1. **Autonomous Vehicles** – Helps vehicles identify road signs automatically.
2. **Advanced Driver Assistance Systems (ADAS)** – Provides warnings to drivers.
3. **Smart Transportation** – Supports intelligent traffic management.
4. **Road Safety Systems** – Helps detect important traffic signs.
5. **Navigation Systems** – Provides additional information about road conditions.
6. **Traffic Monitoring** – Can be used to analyze road images automatically.

19. Future Enhancement

The system can be improved in the future by using advanced deep learning models such as **ResNet, EfficientNet, YOLO, Vision Transformers, or other lightweight architectures**.

The system can also be extended to real-time video recognition using a camera mounted on a vehicle. Additional techniques such as data augmentation and transfer learning can improve performance under different weather and lighting conditions.

The trained model can also be optimized for deployment on mobile devices and embedded systems so that traffic signs can be recognized in real time.

20. Conclusion

The **CNN-Based Traffic Sign Recognition System** demonstrates how deep learning can be applied to computer vision and intelligent transportation systems. CNNs are effective for traffic sign classification because they can automatically learn useful visual features such as edges, shapes, colors, and patterns.

The proposed system consists of image preprocessing, CNN-based feature extraction, classification, and performance evaluation. With a suitable dataset and proper training, the system can recognize different traffic signs accurately. This technology has important applications in autonomous vehicles, driver-assistance systems, and smart transportation.

21. References

1. LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521, 436–444.
2. Stallkamp, J., Schlipsing, M., Salmen, J., & Igel, C. (2012). Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition. *Neural Networks*, 32, 323–332.
3. Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*.
4. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
5. Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. *International Conference on Learning Representations*.
6. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.